| Institución Universitaria | GUÍA DE TRABAJO<br>Tecnología en Desarrollo de Software | Código | FDE 048 |
|---|---|---|---|
| | | Versión | 03 |
| | | Fecha | 2009-06-09 |

## 1. IDENTIFICACIÓN

delioaristizabal@itm.edu.co

| Asignatura | | *Programación de Software* | | | | | Guía No. | 4-1 |
|---|---|---|---|---|---|---|---|---|
| Área | | Básicas de la Tecnología | | | Nivel | | 4 | |
| Código | | **580304006** | | | Pensum | | 1 | |
| Correquisito(s) | | *580506004* | | | Prerrequisito(s) | | NA | |
| Créditos | 4 | TPS | 64 | TIS 128 | TPT | | TIT | |
| **TRABAJO INDEPENDIENTE** | | | | | **TRABAJO PRESENCIAL** | | | |
| Trabajo Teórico | X | Trabajo Práctico | X | | Trabajo Teórico | | Trabajo Práctico | X |

## 2. IDENTIFICACIÓN

| COMPETENCIAS | CONTENIDO TEMÁTICO | INDICADOR DE LOGRO |
|---|---|---|
| Analiza, diseña e implementa aplicaciones de desarrollo de software en capas, utilizando la programación orientada a objetos con énfasis en bases de datos a partir de los requerimientos del cliente, para la solución de problemas en los sistemas de información. | Preferir software para la realización de aplicaciones donde se resuelvan problema o requerimientos solicitados por el cliente de manera modular.<br><br>Uso de SGBD/DBMS[1] e integración a una aplicación que requiere almacenar información de manera recurrente. (Utilizando diferentes metodologías). | Organiza la capa de datos para la interaccionan de los formularios de la aplicación con la información existente en los SGBD, utilizando el lenguaje SQL para la manipulación de esta.<br><br>Plantear diferentes medios de abstracción e interacción entre la información existente en SGBD y el usuario. |

## 3. RECURSOS REQUERIDOS

Sala de sistemas con equipos de cómputo, 1 proyector de video y marcadores varios.
Material bibliográfico

**Programación a 3 capas**
**Transacciones en BD**
**Parte 1 ( form Maestro )**

## 4. PROCEDIMIENTO

### *4.1 Consultar*

1. ¿Qué es una programación a tres capas? Beneficios y Desventajas.
2. ¿En qué consiste la capa de base de datos?
3. ¿Qué es CRUD en programación?
4. ¿En qué consiste el modelo en un servicio API con servicio CRUD?
5. ¿En qué consiste el control en un servicio API con servicio CRUD? ¿Dónde se realiza?
6. ¿Qué hacen los comportamientos Post, Get, Put, Delete en una API con servicio CRUD?
7. ¿Qué es Entity Framework en .Net ?
8. ¿Cuál es la diferencia de IEnumerable e IQueryable? En Entity Framework .Net

### *4.2 Problema para solucionar*

**Super Eventos Colombia SAS** realiza anualmente el **Festival de bandas de Rock *Thunderstruck*** a nivel nacional, donde en cada evento se inscriben bandas de diferentes ciudades del país y cada banda debe tener un nombre, un año de fundación, una ciudad de origen y debe inscribir un mínimo de 1 artista(solitario). Cada artista tiene un nombre, un número de documento con su respectivo tipo de documento, un género (Masculino o Femenino), ciudad de nacimiento y puede pertenecer a una o varias bandas en cada evento. El evento tiene un nombre, el año de realización, la ciudad, la fecha, lugar donde se realiza y valor de cada artista a inscribir. Para la inscripción se debe especificar el evento, la fecha de inscripción, la banda con todos sus integrantes con el instrumento principal (Batería, Bajo, Guitarra, Sintetizador, Otro y Vocalista) que va a tocar y su edad al momento de la inscripción y el representante de dicha banda con su teléfono de contacto. Se desea un sistema CRUD para la transaccionalidad de las ciudades, eventos, bandas, géneros, artistas, instrumentos, artistas por banda, la inscripción de las bandas con sus respectivos artistas. El programa debe tener el sistema de navegación en cada formulario, ya sea maestro u operativo; cada formulario debe tener la forma de poder realizar las opciones de buscar, ingresar, modificar e imprimir. Permitir realizar consultas por evento: bandas participantes, artista por banda y artistas por instrumento. Este año es la primera inscripción a eventos donde por cada artista se cobra 200 mil pesos al evento. El valor de la inscripción se debe actualizar en cada registro en la inscripción, si se retira un artista de una inscripción ya realizada, no se actualiza el valor, se realizará por otro proceso.

[1] Sistema de Gestión de Bases de Datos (en inglés: DataBase Management System)



Powered by CamScanner

## Actividad # 1. Bases de Datos:

1.1. Crear una carpeta con el nombre: ***Practica4***

1.2. Crear una subcarpeta con nombre: ***basedeDatos***

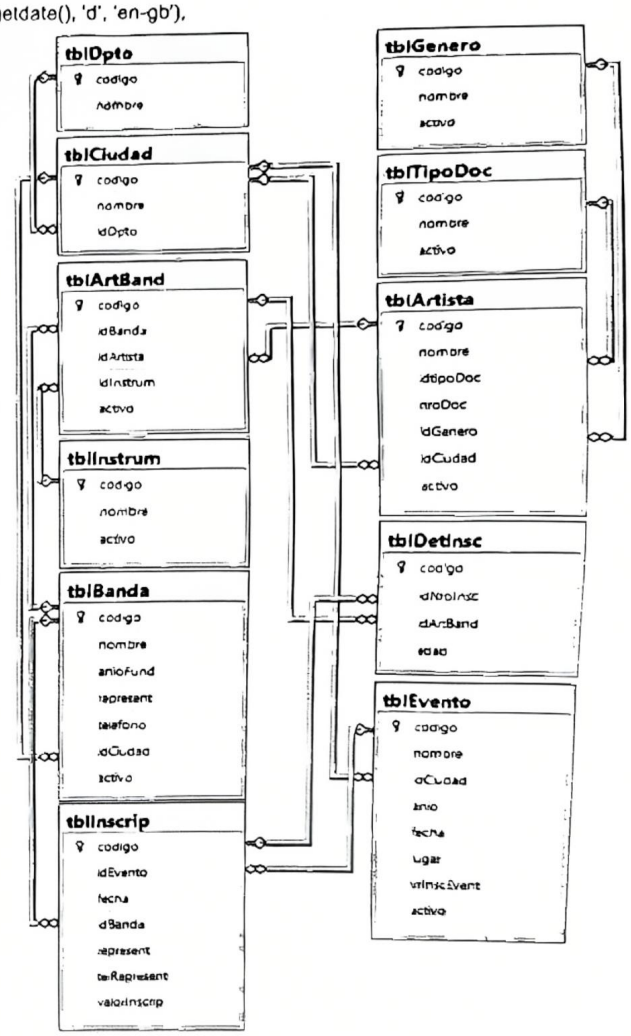
1.3. Crear un script de base de datos en el DBMS SQL Server y guárdalo en la subcarpeta creada en el numeral anterior, dicho script debe contener los siguientes ítems:

i. Crear y usar la base de datos: ***bdEventos*** (colocar la instrucción: ***go*** al final de cada estructura, instrucción o grupo de instrucciones)

ii. Crear las tablas: ***tblTipoDoc, tblGenero, tblInstrum, tblDpto, tblCiudad, tblEvento, tblBanda, tblArtista, tblArtBand, tblInscrip y tblDetInsc.*** *(Entre cada creación de tabla, colocar la instrucción: go)*

```
create DATABASE bdEventos
go
USE bdEventos
go
---------- Tablas ----------
create table tblTipoDoc (
  codigo int primary key,
  nombre varchar(50) Not Null,
  activo bit default 1 Not Null );
go
create table tblGenero (
  codigo int primary key,
  nombre varchar(20) Not Null,
  activo bit default 1 Not Null );
go
create table tblInstrum (
  codigo int primary key,
  nombre varchar(20) Not Null,
  activo bit default 1 Not Null );
go
create table tblDpto (
  codigo int primary key,
  nombre varchar(50) Not Null );
go
create table tblCiudad (
  codigo int primary key,
  nombre varchar(50) Not Null,
  idDpto int  Not Null);
go
create table tblEvento (
  codigo int primary key,
  nombre varchar(50) Not Null,
  idCiudad int Not Null,
  anio int Not Null,
  fecha varchar(30) Not Null,
  lugar varchar(80) Not Null,
  vrInscEvent decimal(9,2) default 0,
  activo bit default 1 Not Null );
go
create table tblBanda (
  codigo int primary key,
  nombre varchar(50) Not Null,
  anioFund int Not Null,
  represent varchar(80) Not Null,
  telefono varchar(25) Not Null,
  idCiudad int Not Null,
  activo bit default 1 Not Null );
go
```

```
create table tblInscrip (
  codigo int primary key,
  idEvento int Not Null,
  fecha varchar(12) default FORMAT(getdate(), 'd', 'en-gb'),
  idBanda int Not Null,
  represent varchar(80) Not Null,
  telRepresent varchar(25) Not Null,
  valorInscrip decimal(12,2) default 0
);
go
create table tblArtista (
  codigo int primary key,
  nombre varchar(80) Not Null,
  idtipoDoc int Not Null,
  nroDoc int Not Null,
  idGenero int Not Null,
  idCiudad int Not Null,
  activo bit default 1 Not Null );
go
create table tblArtBand (
  codigo int identity(1,1) primary key,
  idBanda int Not Null,
  idArtista int Not Null,
  idInstrum int Not Null,
  activo bit default 1 Not Null );
go
create table tblDetInsc (
  codigo int identity(1,1) primary key,
  idNroInsc int Not Null,
  idArtBand int Not Null,
  edad int Not Null );
go
```




**Modelo Relacional**

iii. Crear el modelo relacional entre las tablas (Ver figura anterior) *(al final de las relaciones colocar la instrucción: **go**)*

```
ALTER TABLE tblEvento    add Foreign key (idCiudad)   REFERENCES tblCiudad   (codigo)
ALTER TABLE tblCiudad    add Foreign key (idDpto)     REFERENCES tblDpto     (codigo)
ALTER TABLE tblArtista   add Foreign key (idtipoDoc)  REFERENCES tblTipoDoc  (codigo)
ALTER TABLE tblBanda     add Foreign key (idCiudad)   REFERENCES tblCiudad   (codigo)
ALTER TABLE tblArtista   add Foreign key (idGenero)   REFERENCES tblGenero   (codigo)
ALTER TABLE tblArtista   add Foreign key (idCiudad)   REFERENCES tblCiudad   (codigo)
ALTER TABLE tblArtBand   add Foreign key (idBanda)    REFERENCES tblBanda    (codigo)
ALTER TABLE tblArtBand   add Foreign key (idArtista)  REFERENCES tblArtista  (codigo)
ALTER TABLE tblArtBand   add Foreign key (idInstrum)  REFERENCES tblInstrum  (codigo)
ALTER TABLE tblInscrip   add Foreign key (idEvento)   REFERENCES tblEvento   (codigo)
ALTER TABLE tblInscrip   add Foreign key (idBanda)    REFERENCES tblBanda    (codigo)
ALTER TABLE tblDetInsc   add Foreign key (idNroInsc)  REFERENCES tblInscrip  (codigo)
ALTER TABLE tblDetInsc   add Foreign key (idArtBand)  REFERENCES tblArtBand  (codigo)
go
```




Powered by CamScanner

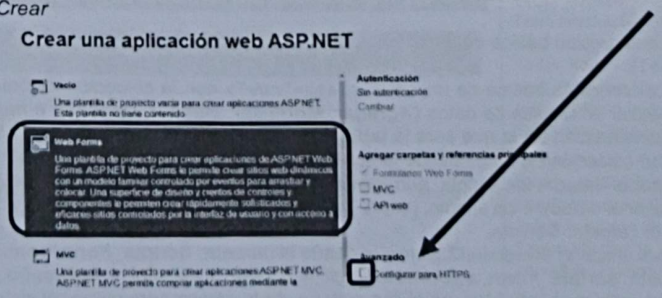
iv. Ingresar datos a las tablas:

```
--------------  tblTipoDoc  ------------
insert into tblTipoDoc values
  ( 1,'Cédula Ciudadanía',1), ( 2,'Pasaporte',1), ( 3,'Cédula de Extranjería',1);
go
--------------  tblGenero  ------------
insert into tblGenero values
  ( 1,'Masculino',1), ( 2,'Femenino',1);
go
--------------  tblInstrum  ------------
insert into tblInstrum values
  ( 101,'Batería',1), ( 102,'Bajo',1), ( 103, 'Guitarra',1), ( 104,'Sintetizador',1), ( 105, 'Otro',1), ( 110, 'Vocalista',1);
go
--------------  tblDpto  ------------
insert into tblDpto values
  ( 110,'Antioquia'), ( 120,'Atlantico'), ( 130, 'Valle'), ( 140, 'Cundinamarca');
go
--------------  tblCiudad  ------------
insert into tblCiudad values
  ( 1, 'Medellín', 110), ( 2, 'Envigado', 110), ( 3, 'Bello', 110),        ( 4, 'Barranquilla', 120), ( 5, 'Soledad', 120),
  ( 6, 'Malambo',120),   ( 7, 'Cali', 130),     ( 8, 'Buenaventura', 130), ( 9, 'Buga',130),          ( 10, 'Bogota', 140),
  (11, 'Usaquen',140),   (12, 'Soacha', 140);
go
--------------  tblEvento  ------------
insert into tblEvento values
  ( 1, 'Rock Medellín 2026', 1, 2026, 'Diciembre 19', 'Estadio Atanasio Girardot', 200000, 1 ),
  ( 2, 'Rock Cali 2027',     1, 2027, 'Junio 26',     'Estadio Pascual Guerrero',  230000, 1 );
go
--------------  tblBanda  ------------
insert into tblBanda values
  ( 1, 'Overdrive', 2016, 'HardRock', '57 (604)2506070', 1, 1 ), ( 2, 'Anankaia', 2017, 'Solo Rock', '57 3003112233', 10, 1 ),
  ( 3, 'Spirit brutal', 2020, 'Miguel Urrutia', '57 3125102030', 4, 1 );
go
--------------  tblArtista  ------------
insert into tblArtista values
  (1, 'Manuela Herrera', 1, 1234, 2, 1, 1), ( 2, 'Manuela Suarez', 1, 5678, 2, 2, 1), ( 3, 'Melina Pinzón', 1, 3876, 1, 7, 1),
  (4, 'Mauricio Cardona', 1, 9890, 1, 8, 1), ( 5, 'Scott Ragun', 3, 1020, 1, 5, 1),   ( 6, 'Jaz Ragun', 3, 3121, 1, 12, 1),
  (7, 'Maya Márquez', 1, 4321, 2, 11, 1),     ( 8, 'Mateo Betancur', 1, 7890, 1, 4, 1), ( 9, 'Alvaro Rojas', 1, 9020, 1, 3, 1),
 (10, 'Nikole Bark', 2, 7080, 2, 6, 1), (11, 'Dave Luna', 3, 8090, 1, 10, 1), (12, 'Daniel A. Aristizábal', 1, 7050, 1, 1, 1);
go
--------------  tblArtBand  ------------
insert into tblArtBand values
  ( 2, 1, 101, 1 ), ( 2, 7, 102, 1 ), ( 2, 3, 103, 1 ),  ( 2, 2, 110, 1 ), ( 2, 12, 103, 1 ),
  ( 1, 1, 110, 1 ), ( 1, 4, 102, 1 ), ( 1, 8, 101, 1 ),  ( 1, 9, 103, 1 ), ( 1, 12, 103, 1 ),
  ( 3, 6, 102, 1 ), ( 3, 5, 103, 1 ), ( 3, 10, 104, 1 ), ( 3, 11, 110, 1 );
go
--------------  tblInscrip  ------------
insert into tblInscrip values
  ( 1, 1, '15/9/2026', 2, 'Alejandro Aristizábal', '310203040', 1000000 );
go
--------------  tblDetalle_Inscrip  ------------
insert into tblDetInsc values
  (1, 1, 27 ),  ( 1, 2, 26 ), ( 1, 3, 26 ), ( 1, 4, 25 ), ( 1, 5, 25 );
go
```

```
select * from tblArtBand
select * from tblInscrip
select * from tblDetInsc
      where idNroInsc = 1
```

**Actividad 2. Crear Proyecto WEB en C# con sistema de navegación**

2.1. En Microsoft Visual Studio 2026, utilizar la opción: ***Crear un Proyecto***, crear un proyecto tipo: *Aplicación web ASP.NET (.NET Framework)* en C#, con nombre: ***webFestRock***, en la subcarpeta creada en la actividad 1.1. *Aceptar.*
Te lleva a la ventana de *Crear una aplicación web ASP.NET*, en ella seleccionar: ***Web Forms*** y ***deschequear*** la opción: configurar para HTTPS, luego: *Crear*



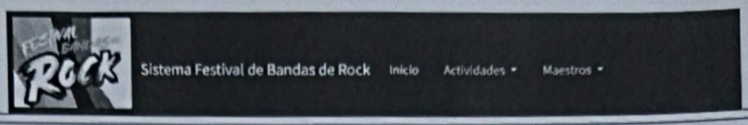

2.2. Crear una carpeta llamada: ***Imagenes,*** en ella adicionar las imágenes suministradas por el docente.
2.3. Agregar la carpeta: **páginas**, suministrada por el docente al proyecto.
2.4. Agregar la carpeta: **Scripts**, suministrada por el docente sobrescribiendo lo que se tiene originalmente.
2.5. De la carpeta: Paginas, modificar la página: **menu.html**
   i. Establecer la imagen: (***LogoFestRock.jpg***) como logo del menú



Powered by CamScanner

ii. Reemplazar el texto: Nombre Proyecto por: **Sistema Festival de Bandas de Rock**.
iii. Crear las opciones de navegación en el navbar, acorde a las siguientes imágenes. (9 opciones de navegación), Reemplazar **Menú 1** por **Inscripción**, luego copiar dicha línea y pegarla a continuación con nombre: **Artista por banda** y Reemplazar: **Menú 2** por **Maestros**

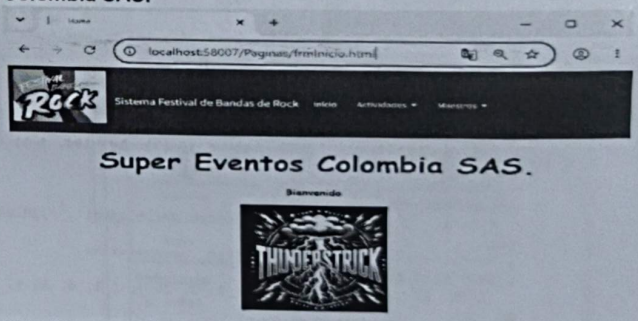
**Vista General**




| **Inicio y Actividades** | **Maestros** | |
|---|---|---|
| **Inicio**: | **Opciones** | |
| Opción | Artista = | href = frmArtista.html |
| Inicio: href = frmInicio.html | Evento = | href = frmEvento.html |
| **Actividades**: | Instrumento = | href = frmInstrum.html |
| Opción: | Género = | href = frmGenero.html |
| Inscripción, href = frmInscrip.html | Ciudad = | href = frmCiudad.html |
| Artista por banda, href = frmArtXBand.html | Departamento = | href = frmDpto.html |
| | Tipo Documento = | href = frmTipDoc.html |

2.6. **AdminLTE 3 | Dashboard**
i. Visitar la página: https://adminlte.io/themes/v3/ (visitada julio 27/26), ver los formatos **adminLTE** como Dashboard libre
ii. Observar los diferentes objetos tipo HTML, ver opción: **Forms --> General Elements** y **Advances Elements**

2.7. Modificar la página: **frmInicio.html** de la carpeta: **Paginas**, acorde a la siguiente imagen.
i. Colocar el título <title>: Home – **Sist. Festival de Bandas.**
ii. En el encabezado <head>: desde la carpeta: **Estilos** referenciar el script: adminlte.min.css y desde la carpeta: **Estilos/css** el script: all.min.css. <!--Para imágenes botones-->.
iii. Reemplazar: **Texto para Título** por el texto: **Super Eventos Colombia SAS.**
iv. Copiar el anterior  con el texto: **Bienvenido**, con un size = 4
v. Establecer la imagen: (***FestRock.jpg***) como imagen central.
vi. Desde la carpeta: **Scripts,** antes de finalizar el </body>: referenciar los scripts: jquery-3.7.0.min.js y bootstrap.bundle.min.js (Conservando el mismo orden).
vii. Desde la carpeta: **Scripts_Form**, adicionar el script de control JavaScript: **frmInicio.js** el cual contiene el JQuery (function) que incrusta la página: **Menu.html**, en el div: dvMenu, de la página para utilizar el sistema de navegación.
viii. ***Establecer como página de Inicio el form:*** frmInicio.html

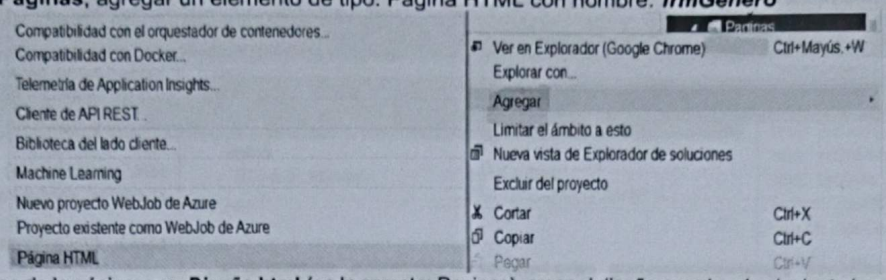



2.8. Página: **paraDiseño.html** de la carpeta: **Paginas**
i. Mirar la página: **PaginaBase.html** de la carpeta: **Paginas**, con un compendio base del uso de algunos objetos utilizados en las páginas web utilizando el **adminLTE**.
ii. Observar el espacio para colocar el **Título** y el **texto informativo** sobre el maestro/Operativo que se manejará en el formulario.
iii. Observar la construcción básica de un `<div class="row">` con 2 objetos de captura (`input/text`) con sus respectivas etiquetas (`label`).
iv. Observar la construcción básica de un `<div class="row">` con 1 objeto de chequeo (`input/checkbox`) con su respectiva etiqueta (`label`).
v. Observar la construcción básica de un `<div class="row">` con la colección de botones (`button`) para el control de las acciones a realizar en la base de datos (***Agregar***, ***Modificar***, ***Buscar***, ***Cancelar*** e ***Impresión***).
vi. Observar la construcción de lo que será la tabla/grid (***tablaDatos***) con los títulos de columna con la información que será mostrada como colección de datos desde la base de datos en el objeto de tipo: `table`.
vii. En el encabezado <head>: los scripts: adminlte.min.css y all.min.css, desde la carpeta: **Estilos**
viii. Antes de finalizar el </body>: los scripts: jquery-3.7.0.min.js, bootstrap.bundle.min.js y bootstrap.min.js (Conservando el mismo orden) desde la carpeta: **Scripts**.
ix. Para las alertas el script: sweetalert2.all.min.js desde la carpeta: **Scripts_Form/Comunes**.
x. Desde la carpeta: **Scripts_Form**, adicional el script de control JavaScript: **paraDiseño.js** el cual contiene el JQuery (function) para incrustar la página: Menu.html, en el div: dvMenu, de la página para utilizar el sistema de navegación. (En cada página se debe de cambiar al archivo de control de dicha página HTML)
xi. Ejecutar el proyecto para observa su diseño inicial y funcionamiento. Navegación y forms: **Inicio** y **paraDiseño**

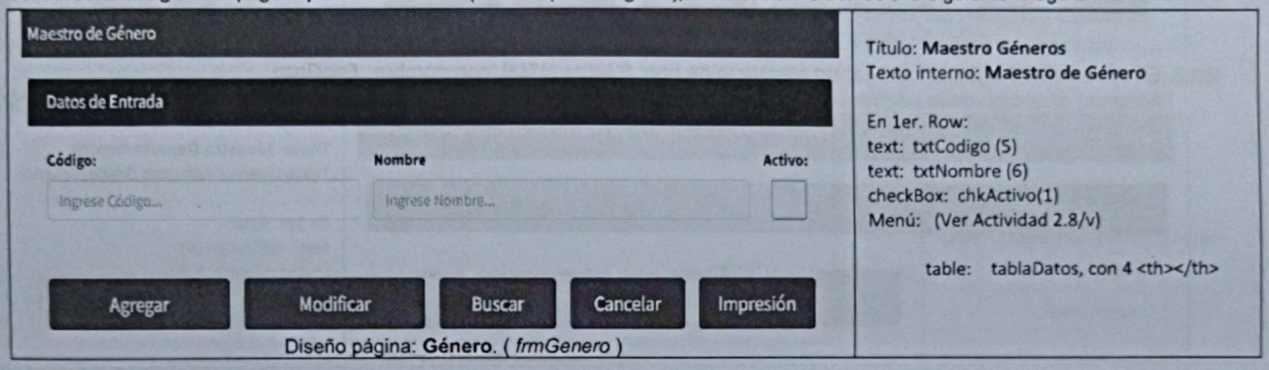


Powered by CamScanner

2.9. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmGenero***



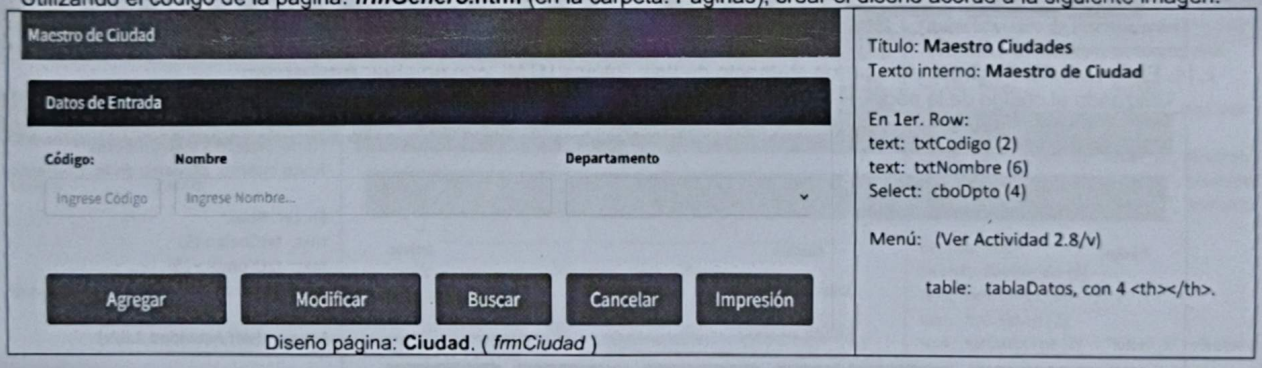

Utilizando el código de la página: ***paraDiseño*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Diseño página: **Género**. ( *frmGenero* )

2.10. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmCiudad***

Utilizando el código de la página: ***frmGenero*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Diseño página: **Ciudad**. ( *frmCiudad* )

2.11. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmArtista***

Utilizando el código de la página: ***frmCiudad*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.

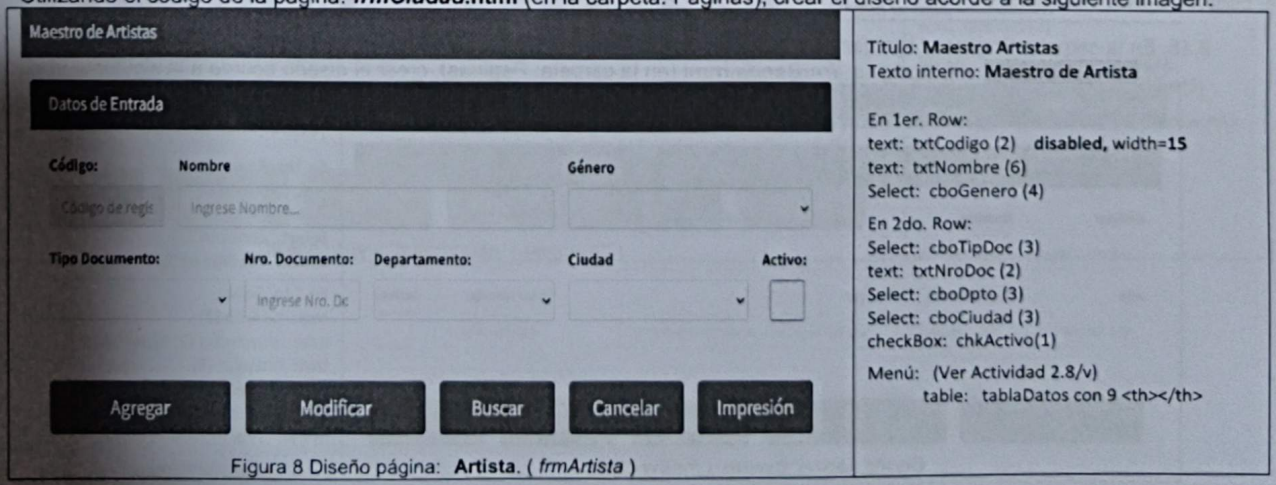
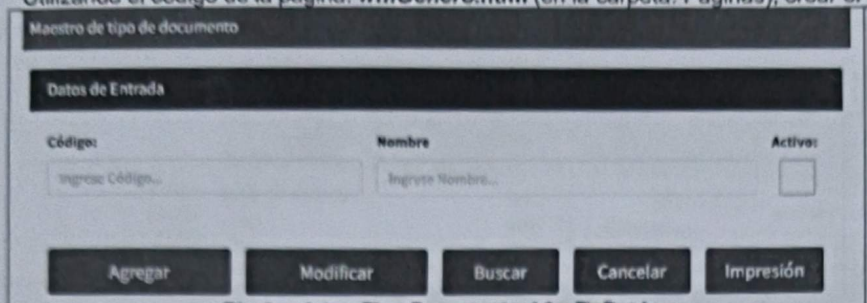



Figura 8 Diseño página: **Artista**. ( *frmArtista* )



Powered by CamScanner

2.12. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmTipDoc***
Utilizando el código de la página: ***frmGenero*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro Tipo Documentos**
Texto interno: **Maestro tipo de documento**

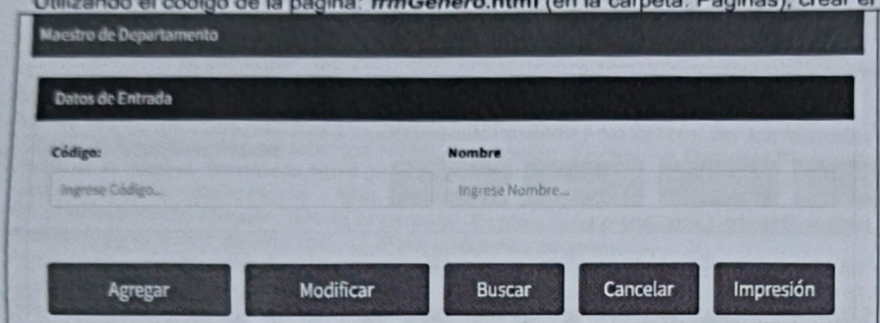
En 1er. Row:
text: txtCodigo (5)
text: txtNombre (6)
checkBox: chkActivo(1)

Menú: (Ver Actividad 2.8/v)

table: tablaDatos, con 4 <th></th>

Diseño página: **Tipo Documento**. ( *frmTipDoc* )

2.13. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmDpto***
Utilizando el código de la página: ***frmGenero*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro Departamentos**
Texto interno: **Maestro Departamento**

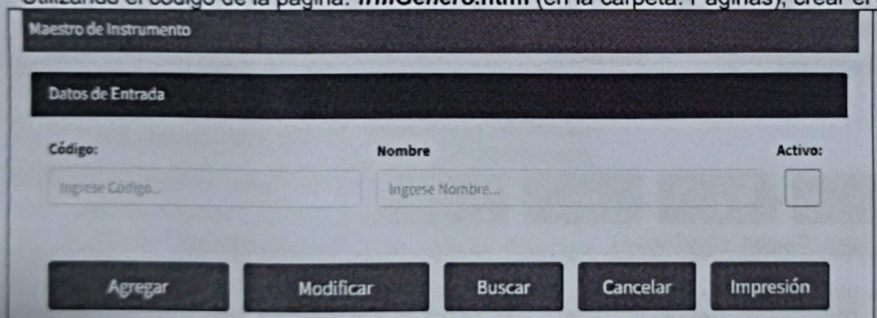
En 1er. Row:
text: txtCodigo (6)
text: txtNombre (6)

Menú: (Ver Actividad 2.8/v)

table: tablaDatos, con 3 <th></th>

Diseño página: **Departamento**. ( *frmDpto* )

2.14. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmInstrum***
Utilizando el código de la página: ***frmGenero*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro Instrumentos**
Texto interno: **Maestro de Instrumento**

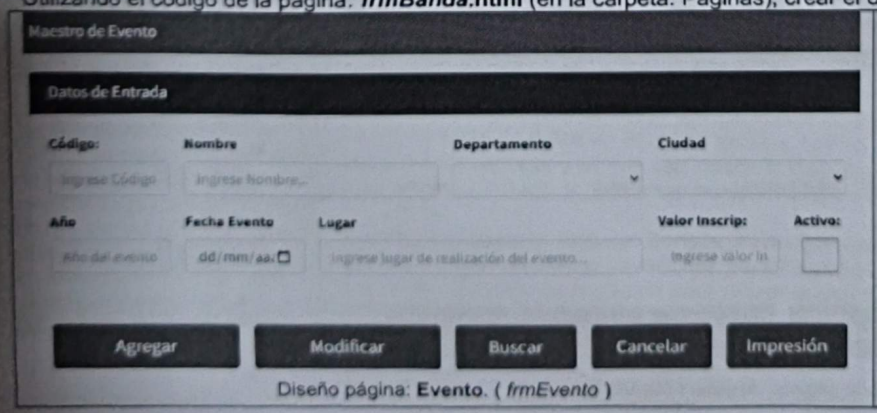
En 1er. Row:
text: txtCodigo (5)
text: txtNombre (6)
checkBox: chkActivo(1)

Menú: (Ver Actividad 2.8/v)

table: tablaDatos, con 4 <th></th>

Diseño página: **Instrumento**. ( *frmInstrum* )

2.15. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmEvento***
Utilizando el código de la página: ***frmBanda*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro de Eventos**
Texto interno: **Maestro Evento**

En 1er. Row:
text: txtCodigo (2)
text: txtNombre (4)
Select: cboDpto (3)
Select: cboCiudad (3)

En 2do. Row:
text: txtAnio (2)
date: dtmFecha (2) type="**date**"
text: txtLugar (5)
text: txtValor (2)
checkBox: chkActivo(1)

Menú: (Ver Actividad 2.8/v)

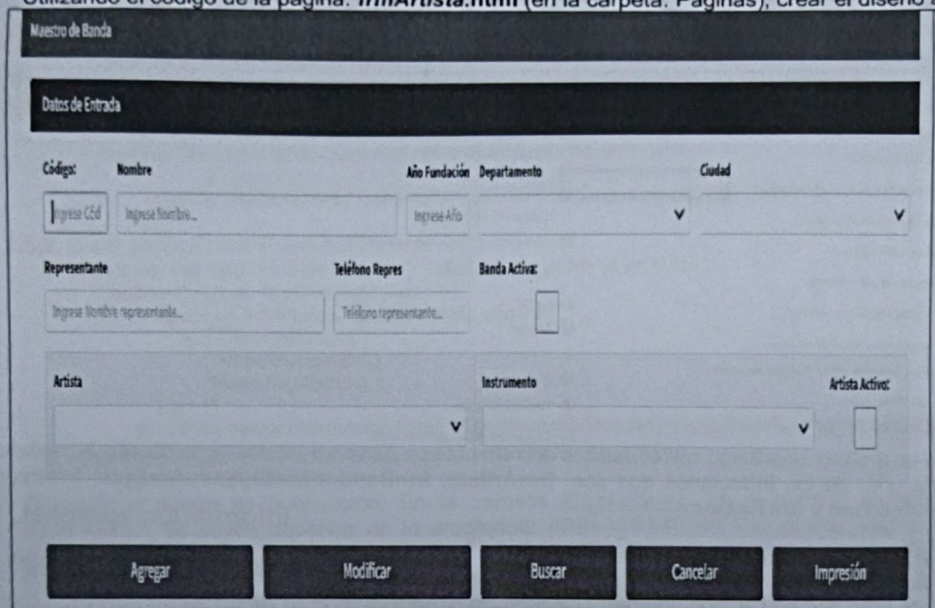
table: tablaDatos, con 10 <th></th>

Diseño página: **Evento**. ( *frmEvento* )



Powered by CamScanner

2.16. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmBanda***
Utilizando el código de la página: ***frmArtista*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro de Bandas**
Texto interno: **Maestro de Banda**

En 1er. Row:
text: txtCodigo (1)
text: txtNombre (4)
text: txtAnio (1)
Select: cboDpto (3)
Select: cboCiudad (3)

En 2do. Row:
text: txtRepres (5)
text: txtTelRep (2)
checkBox: chkActivo(1)

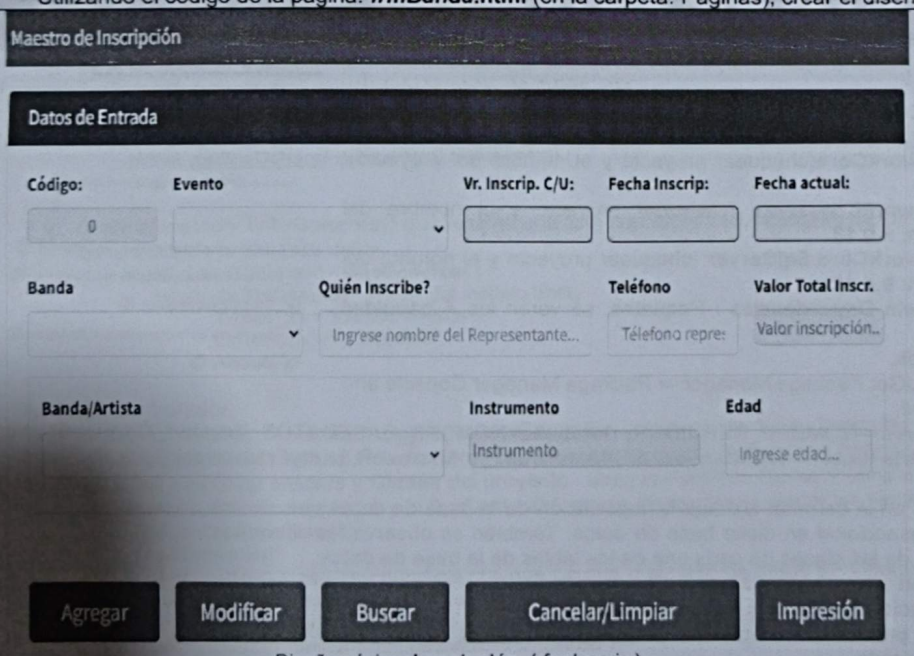
3er. Row: table: tblDatosDetalle (12)
Select: cboArtBand (50%)
text: cboInstrum (30%)
checkBox: chkActArt(20%)

Menú: (Ver Actividad 2.8/v)

table: tablaDatos, con 8 <th></th>

Diseño página: **Banda**. ( *frmBanda* )

2.17. En la carpeta: **Paginas**, agregar un elemento de tipo: Página HTML con nombre: ***frmInscrip***
Utilizando el código de la página: ***frmBanda*.html** (en la carpeta: Paginas), crear el diseño acorde a la siguiente imagen.




Título: **Maestro de Inscripciones**
Texto interno: **Maestro de Inscripción**

En 1er. Row: class=
text: txtCodigo (2) **"label_1"**, disabled
Select: cboEvento (4)
Label: lblVrIndInsc (2) **"label_2"**, disabled
Label: lblFechaInsc (2) **"label_2"**, disabled
Label: lblFechaAct (2) "label_2", disabled

En 2do. Row:
Select: cboBanda (4)
text: txtRepres (4)
text: txtTelRep (2)
text: txtValorTot (2) **"label_1"**, disabled

En 3er. Row: table: tblDatosDetalle (12)
Select: cboArtBand (6)
text: txtInstrum (4) **"label_1"**, disabled
text: txtEdad (2)

Menú: btnAgre(2), btnModi(2), btnBusc(2), btnCanc(4) y btnImpr(2)

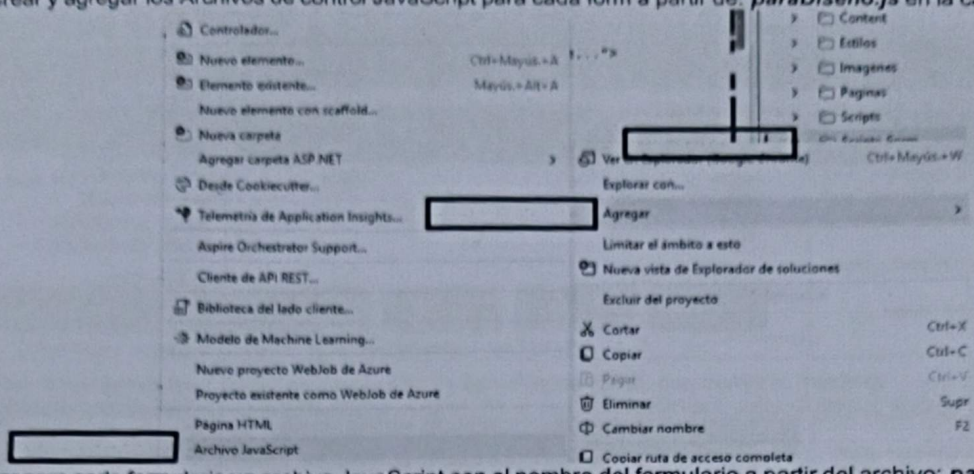
table: tablaDatos, con 8 <th></th>

Diseño página: **Inscripción**. ( *frmInscrip* )



Powered by CamScanner

2.19. Crear y agregar los Archivos de control JavaScript para cada form a partir de: ***paraDiseño.js*** en la carpeta: **Scripts**




Crear para cada formulario un archivo JavaScript con el nombre del formulario a partir del archivo: ***paraDiseño.js***, replicando el mismo código para cada uno. de los formularios que son: **frmArtista, frmBanda, frmCiudad, frmDpto, frmEvento, frmGenero, frmInscrip, frmInstrum y frmTipDoc.**

**Actividad 3.** Aplicación API WEB para realizar el servicio Web API

3.1. En Microsoft Visual Studio 2026, utilizar la opción: ***Crear un proyecto***, crear un proyecto tipo: *ASP.NET Core Web API* en C#, con nombre: ***apiFestRock***, en la carpeta de la práctica. *Aceptar*. Te lleva a la ventana con la información adicional (***deschequear*** el checkBox, ***Configurar para HTTPS***), luego: *Crear*.

3.2. Quitar el archivo: **WeatherForecast**.cs de la raíz del proyecto y el archivo: **WeatherForecastController**.cs de la carpeta: **Controller**

3.3. select menu Tools -> **NuGet Package Manager** -> administrador de paquetes Nuget para solución. Seleccionar la pestaña: Examinar, en ella en el textbox Buscar e instalar los siguientes paquetes:

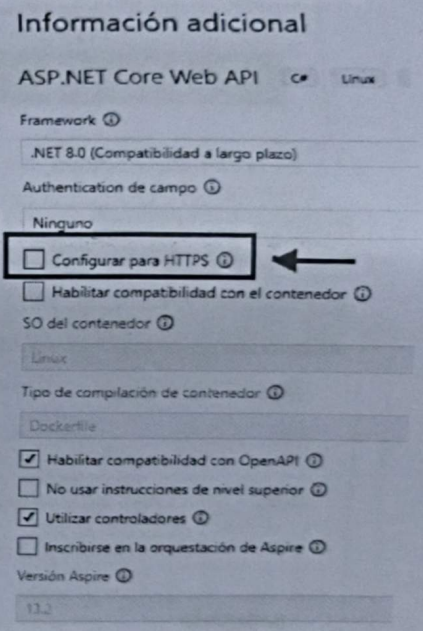
3.3.1. **Swashbuckle.AspNetCore**, (chequear proyecto y el nombre del proyecto), seleccionar versión: **6.6.2 (** Verificar antes sino se encuentra pre-instalado **)**

3.3.2. **Microsoft.EntityFrameworkCore**(chequear proyecto y el nombre del proyecto), seleccionar versión: **9.0.18.**

3.3.3. **Microsoft.EntityFrameworkCore.tools** (chequear proyecto y el nombre del proyecto), seleccionar versión: **9.0.18.**

3.3.4. **Microsoft.EntityFrameworkCore.SqlServer** (chequear proyecto y el nombre del proyecto), seleccionar versión: **9.0.18.**

Mirar que, en el proyecto, carpeta Dependencias / Paquetes, se verán los 4 paquetes instalados anteriormente.

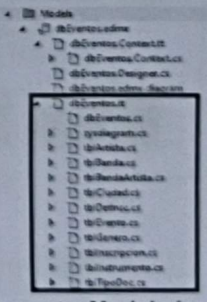



3.4. Migración de la base de datos.

3.4.1 select menu Tools -> NuGet Package Manager -> Package Manager Console and run the following command:

Scaffold-DbContext **"Server**=NOMBRE_SERVIDOR; **Database**=NOMBRE_BASEDATOS; **Trusted_Connection**=true; **MultipleActiveResultSets**=true; **TrustServerCertificate**=true;" **Microsoft.EntityFrameworkCore**.SqlServer **-OutputDir Models**

Observar que se incrusta en la carpeta: **Models** la importación de la base de datos que servirá para realizar la parte transaccional en dicha base de datos. También se observa los diferentes archivos con los modelos de las clases de cada una de las tablas de la base de datos.



3.4.2. En la carpeta *Models* en el archivo de clase, ***BdEventosContext***.

- ✓ **Borrar** el constructor vacío, solo dejar el parametrizado
- ✓ **Comentar** el método: protected override void OnConfiguring, luego se necesitará la cadena de conexión de dicho método.
- ✓ **Borrar** las relaciones entre las tablas en la clase
  - ➢ De cada tabla eliminar los párrafos que tenga el siguiente inicio **entity.HasOne**; por ejemplo, de la tabla: **TblArtBand** contiene 3 párrafos, el 1ro. inicia así: **entity.HasOne**(d => d.IdArtistaNavigation).WithMany(p => p.TblArtBands). En total deben ser 13, o sea la misma cantidad de relaciones entre tablas
  - ➢ Borrar o comentar de cada tabla todos los métodos virtuales (de relaciones o ICollection) de la carpeta: **Models** (que solo queden los campos/propiedades). Por ejemplo, de la tabla **TblArtBand**. // **public virtual** TblArtistum

3.5. En el archivo: ***appsettings.json***. Copiar la cadena de conexión que se encuentra en el archivo del context (BdEventosContext) de la carpeta: **Models**. En el método comentado OnConfiguring. Agregar el argumento "ConnectionStrings", colocando una coma después de: "AllowedHosts": "*" la siguiente línea de propiedad:



Powered by CamScanner

```
"ConnectionStrings" : {  "cnx" :
  "Server=Nombre_Server;      Database=Nombre_BD;      Trusted_Connection=True;      MultipleActiveResultSets=true;
TrustServerCertificate=True;"
}
```

**cnx**: es el nombre de la conexión entre la aplicación y la base de datos.

3.6. Para reconocer la cadena de conexión y **habilitar el Cors**. En el archivo: ***Program.cs***

3.6.1. **Agregar *después*** de: var builder = WebApplication.CreateBuilder(args);

```
//Con el nombre del context del archivo: models
builder.Services.AddDbContext<BdEventosContext>( options =>
  {
    options.UseSqlServer( builder.Configuration.GetConnectionString( "cnx" ) );
  });
```

3.6.2. Crear política para el uso del Cors genérico.

a. **Antes** de: var app = builder.Build( ); colocar la siguiente política

```
builder.Services.AddCors(opciones => {
    opciones.AddPolicy( "politicaGral", app => {
      app.AllowAnyOrigin( )
         .AllowAnyHeader( )
         .AllowAnyMethod( );
    } );
  }); // Para mayor información mirar: https://emrebener.medium.com/configuring-cors-in-asp-net-core-e985a9b4c0a1
```

b. **Antes** de: app.UseAuthorization(); colocar la siguiente política

```
app.UseCors("politicaGral");
```

3.7. Dirección o puerto de publicación. En la carpeta *Propiedades*, en el archivo ***launchSettings.json***, en la sección: `"profiles"`, se puede observar en la propiedad: `applicationUrl` el puerto asignado inicialmente al localhost del proyecto, se puede cambiar al deseado. Y en `"profiles"`, el puerto asignado para el `swagger`.

**Actividad 4. Acciones CRUD para formulario: frmArtista**

4.0. Crear la carpeta: **Clases** en el proyecto

4.1. En el archivo API, Acciones para llenar combos

4.1.1 Combo: **Genero**

a. Crear clase operativa.

i. En la carpeta: **Clases**, crear (agregar) la clase: clsOpeGenero

ii. Usar la carpeta: **Models** del proyecto. using apiFestRock.Models;

iii. Crear el atributo de tipo context de la base de datos: **ctx**:

```
private readonly BdEventosContext ctx;
```

iv. Crear el constructor con un parámetro de tipo context de la base de datos llamado, **cntxt**.

```
public clsOpeGenero( BdEventosContext cntxt) {
  ctx = cntxt;
}
```

v. Crear el método: **listaGeneros**() que retorna una lista con los id y los nombres de los géneros que se encuentran activos en la tabla de la base de datos.

```
public List<TblGenero> listaGeneros( ) {
  return ctx.TblGeneros  //Tabla del modelo Entity
        .Where( x => x.Activo == true )
        .  OrderBy( x => x.Nombre )
        .ToList( );
}
```

b. Crear controlador.

i. Sobre la carpeta ***Controllers***, agregar un Controller y que se de tipo: *Controller de MVC: en blanco*, con nombre: **genero**Controller (*cambiar el texto Home por **genero**, no borrar la palabra Controller*).

ii. Usar las carpetas: ***Models*** y ***Clases*** del proyecto. using apiFestRock.Clases; y using apiFestRock.Models;

iii. Colocar la ruta del controlador y el ApiController, **antes de la clase** colocar las siguientes instrucciones:

```
[Route("api/[controller]")][2]
[ApiController][3]
```

iv. Crear el **atributo** privado con nombre: *ctx*, que apunta a la base de datos EF Context:

```
private readonly BdEventosContext ctx;
```

v. Crear el constructor con un parámetro de tipo context llamado, **context**.

```
public generoController( BdEventosContext context ) {
  ctx = context;
}
```

vi. Crear el servicio **Get** asincrónico **HttpGet** con alias: **"listadoGeneros"** con nombre **"listarGeneros"** con retorno tipo: List.

```
[ HttpGet( "listadoGeneros" ) ]
public List<TblGenero> listarGeneros( ) {
  clsOpeGenero oX = new clsOpeGenero( ctx );
```

[2] **[Route("api/[controller]")]** Es un atributo que define la plantilla de ruta para un controlador de la API. Básicamente, le dice a la aplicación cómo construir la URL para acceder a las acciones (métodos) dentro de ese controlador.

[3] **[ApiController]** Se aplica a las clases de controlador para habilitar características y comportamientos específicos de las API web, como el enrutamiento de atributos, la validación automática del modelo y la inferencia de parámetros. En esencia, simplifica el desarrollo de servicios RESTful al automatizar tareas comunes.



Powered by CamScanner

```
        return oX.listaGeneros( );
    }
```

c. Ejecutar el proyecto y probar el servicio con el software: **Swagger** que se abre al ejecutar la API (mirar link).

4.1.2 Combos: **tipo de documento y departamento**

a. Crear clase operativa.
   i. En la carpeta: **Clases**, crear (agregar) las clases: clsOpeTipDoc y clsOpeDpto.
   ii. Usar la carpeta: **Models** del proyecto; agregar las instrucciones Route y ApiController, crear el atributo de tipo context de la base de datos: **ctx**, crear el constructor similar a lo creado en el numeral anterior.
   iii. Crear los métodos: **listaTipDocs**() y **listaDptos**() que retornaran una lista con los id y los nombres de los tipos de documentos que se encuentran activos y de los departamentos en las tablas respectivas en la base de datos.

b. Crear controlador.
   i. Sobre la carpeta ***Controllers***, agregar los Controllers: **tipoDoc**Controller y **departamento**Controller.
   ii. En cada uno adicionar los using a las carpetas clases y Models, adicionar las líneas del Route y ApiController, Adicionar el atributo y el constructor
   iii. Crear los servicios **Get** asincrónico **HttpGet** con alias: **listadoTipDocs** y **listadoDptos** con nombres: **listarTiposDoc** y **listarDptos** con retorno tipo: List en los controladores respectivos; similar a lo visto en el numeral anterior.

c. Ejecutar el proyecto y probar el servicio con el software: **Swagger** que se abre al ejecutar la API.

4.2. En el archivo **HTML** del formulario: **frmArtista**

a. En la sección del Head, observar los scripts de diseño (CSS) y control.

```
<link href="../Estilos/select2.min.css" rel="stylesheet" />                    [Estilos]
<link href="../Estilos/select2-bootstrap4.min.css" rel="stylesheet" />         [Estilos]
```

b. Al final de la página antes del fin del cuerpo **</body>**, observar los scripts de control(js) y llenado del combo:

```
<script src="../Scripts/select2.min.js"></script>                              [Scripts]
<script src="../Scripts_Form/Comunes/Combos.js"></script>                      [Scripts_Form/Comunes]
```

4.3 En el archivo ***JavaScript*** del formulario

i. En el encabezado del archivo (antes de jQuery), asignar valor a la variable: dir con la dirección del servicio de la API:

```
var dir = "http://localhost:XXXXX/api/";   // Sería reemplazar las X por el número del puerto de la ejecución de la API.
```

ii. Después del JQuery, crear la function asincrónica para el llenado del combo de género

```
async function llenarComboGenero ( ) {
  let url = dir + "genero/listadoGeneros";                    //Invoca del servicio el método con el alias
  let rpta = await llenarComboGral( url, "#cboGenero");
  if ( rpta != "Termino" ) {
     alerta("error en llenado combo género", "Error en procedimiento");
     return;
  } }
```

iii. De igual forma crear los métodos: **llenarComboTipDoc ( )** y **llenarComboDpto ( )** acorde al punto anterior.

iv. Adicionar la invocación de los métodos: **llenarComboGenero( )**, **llenarComboTipDoc( )** y **llenarComboDpto( )** dentro del **jQuery(function( )** después de los listener de los botones.

v. Probar funcionamiento del llenado de los combos.

4.4. Llenado de combo anidado: ***Ciudad*** desde combo Departamento

a. Crear clase operativa.
   i. En la carpeta: **Clases**, crear (agregar) la clase: **clsOpeCiudad**
   ii. Usar la carpeta: **Models** del proyecto. using apiFestRock.Models;
   iii. Crear el atributo de tipo context de la base de datos: **ctx**:

```
private readonly BdEventosContext ctx;
```

   iv. Crear el constructor con un parámetro de tipo context de la base de datos llamado, **cntxt**.

```
public clsOpeGenero( BdEventosContext cntxt ) {
   ctx = cntxt;
}
```

   v. Crear el método: ***listaCiudadesXDpto***( ) que retorna una lista con los id y los nombres de las ciudades que se pertenecen a un departamento acorde a su código de la tabla ciudades en la base de datos.

```
public List<TblCiudad> listaCiudadesXDpto( int idDpto ) {
   return ctx.TblCiudads  //Tabla del modelo Entity
      .OrderBy(x => x.Nombre)
      .Where(x => x.IdDpto == idDpto)
      .ToList();
}
```

b. Crear controlador.
   i. Sobre la carpeta ***Controllers***, agregar un Controller, de tipo: *Controller de MVC: en blanco*, con nombre: **ciudad**Controller (*cambiar el texto Home por* ***ciudad***, *no borrar la palabra Controller*).
   ii. Crear el **atributo** privado con nombre: ctx, que apunta a la base de datos EF Context:

```
private readonly BdEventosOrigContext ctx;
```

   iii. Usar las carpetas: ***Models*** y ***Clases*** del proyecto. using apiFestRock.Clases; y using apiFestRock.Models;
   iv. Colocar la ruta del controlador y el ApiController, antes de la clase colocar las siguientes instrucciones:

```
[Route("api/[controller]")]
[ApiController]
```

   v. Crear el atributo de tipo context: **ctx**:

```
private readonly BdEventosContext ctx;
```



Powered by CamScanner

vi. Crear el constructor con un parámetro de tipo context llamado, **context**.

```
public ciudadController( BdEventosContext context ) {
  ctx = context;
}
```

vii. Crear el servicio **Get** asincrónico **HttpGet** con alias: **"listadoCiudsXDpto"** con nombre: **listarCiudadesXDpto** con retorno tipo: List, que recibe como parámetro de tipo entero el código del departamento (codDpto).

```
[ HttpGet( "listadoCiudsXDpto" ) ]
public List<TblCiudad> listarCiudadesXDpto( int codDpto )
{
  clsOpeCiudad oX = new clsOpeCiudad( ctx );
  return oX.listaCiudadesXDpto( codDpto );
}
```

c. Ejecutar el proyecto y probar el servicio con el software: **Swagger** que se abre al ejecutar la API (mirar link).

4.5. En el archivo **JavaScript** del formulario

i. Crear la function asincrónico **llenarComboCiudad**( *idCiu* ) para el llenado del combo de las ciudades por departamento y/o selección de una ciudad especifica acorde al código ingresado como parámetro, *idCiu*

```
async function llenarComboCiudad( idCiu ) {
  let idDpto = $("#cboDpto").val();
  let url = dir + "ciudad/listadoCiudsXDpto?codDpto=" + idDpto;
  let rpta = await llenarComboGral( url, "#cboCiudad" );
  if ( rpta != "Termino" ) {
    alerta( "Error en llenado combo ciudad", "Error en procedimiento" );
    return;
  }
  else if (idCiu != undefined) {
    $("#cboCiudad").val(idCiu);
  }
}
```

ii. Modificar la function asincrónica **llenarComboDpto** para llenar el combo de ciudades de forma anidada, después del llenado del combo de Departamentos adicionar el siguiente código

```
llenarComboCiudad( ); //Después del condicional
```

iii. Crear listener para el botón **Cancelar**. Crear la función sincrónica Cancelar

```
function Cancelar() {
  $("#txtCodigo").val("0");
  $("#txtNombre").val("");
  $("#cboGenero")[0].options[0].selected = true;
  $("#cboTipDoc")[0].options[0].selected = true;
  $("#txtNroDoc").val("");
  $("#cboDpto")[0].options[0].selected = true;
  llenarComboCiudad();
  $("#chkActivo").prop("checked",true);
  $("#txtNombre").trigger("focus");
}
```

iv. Invocar el listener para el botón Cancelar. En la opción: $("**#btnCanc**").on("click", invocar la function asincrónica: **Cancelar**( ).

4.2.7. En el archivo **HTML** del formulario para invocar el llenado anidado de las ciudades a partir de la selección de un departamento en el combo.

i. Modificar la declaración del combo de departamento (*cboDpto*), adicionando dentro del tag el evento: **onchange**

```
onchange="llenarComboCiudad( );"
```

ii. Probar funcionamiento hasta el momento.

5. **Artista**: Servicios de búsqueda y transaccionales.

5.1. Procesos generales para la clase y el controlador

a. Crear clase operativa y Controlador en el proyecto API.

i. En la carpeta: **Clases**, crear (agregar) la clase: **clsOpeArtista**

ii. Usar la carpeta: **Models** del proyecto. using apiFestRock.Models;

iii. Crear el atributo de tipo context de la base de datos: **ctx**:

```
private readonly BdEventosContext ctx;
```

iv. Crear el constructor con un parámetro de tipo context de la base de datos llamado, **cntxt**.

```
public clsOpeArtista( BdEventosContext cntxt) {
  ctx = cntxt;
}
```

v. Sobre la carpeta ***Controllers***, agregar un Controller, de tipo: *Controller de MVC: en blanco*, con nombre: **artista**Controller (*cambiar el texto Home por **artista**, no borrar la palabra Controller*).

vi. Crear el **atributo** privado con nombre: *ctx*, que apunta a la base de datos EF Context:

```
private readonly BdEventosOrigContext ctx;
```

vii. Usar las carpetas: ***Models*** y ***Clases*** del proyecto. using apiFestRock.Clases; y using apiFestRock.Models;

viii. Colocar la ruta del controlador y el ApiController, antes de la clase colocar las siguientes instrucciones:

```
[Route("api/[controller]")]
[ApiController]
```

ix. Crear el atributo de tipo context: ***ctx***:

```
private readonly BdEventosContext ctx;
```



Powered by CamScanner

x. Crear el constructor con un parámetro de tipo context llamado, **context.**

```
public artistaController( BdEventosContext context ) {
  ctx = context;
}
```

**5.2. Búsqueda y llenado de tabla**

5.2.1. Procedimientos en la API

a. En la clase operativa, crear el método para buscar la información de un artista acorde a su número de documento.

i. Crear el método privado **artistaXNroDoc**, que tiene como parámetro el número de documento (**nroDocu**) y retorne los datos de tipo **IQueryable** que sirvan para mostrar en el formulario.

```
public IQueryable artistaXNroDoc ( int nroDocu ) {
  return from tA in ctx.Set<TblArtistum>( )
       join tC in ctx.Set<TblCiudad>( ) on tA.IdCiudad equals tC.Codigo
       where tA.NroDoc == nroDocu
       select new {
         codigo = tA.Codigo,
         nombre = tA.Nombre,
         idTD = tA.IdtipoDoc,
         nroDoc = tA.NroDoc,
         idG = tA.IdGenero,
         idDep = tC.IdDpto,
         idC = tA.IdCiudad,
         act = tA.Activo
       };
}
```

ii. Crear el método privado **listarArtistas**, sin parámetro y retorne de tipo IQueryable los datos de todos los artistas registrados que sirven para llenar la tabla en el formulario.

```
public IQueryable listarArtistas() {
  return from tA in ctx.Set<TblArtistum>()
       join tTd in ctx.Set<TblTipoDoc>()  on tA.IdtipoDoc equals tTd.Codigo
       join tG in ctx.Set<TblGenero>()    on tA.IdGenero equals tG.Codigo
       join tC in ctx.Set<TblCiudad>()    on tA.IdCiudad equals tC.Codigo
       join tD in ctx.Set<TblDpto>()      on tC.IdDpto equals tD.Codigo
     orderby tA.NroDoc
     select new          //Ver icons,  https://www.w3schools.com/icons/icons_reference.asp#gsc.tab=0
     {                   //Es doble comillas simples
       editar    = "<a class='btn btn-info btn-sm' href=''><i class='fa fa-search-plus'></i> ir </a>",
       codigo = tA.Codigo,
       nombre = tA.Nombre,
       tipo_Doc = tTd.Nombre,
       nro_Doc = tA.NroDoc,
       genero = tG.Nombre,
       dpto = tD.Nombre,
       ciudad = tC.Nombre,
       activo = tA.Activo
     }; // Los 9 <th></th> para la tabla
}
```

b. En el controlador, crear el servicio **HttpGet** con alias: **consultaArtista** con retorno tipo: **IQueryable** con método: **Get** con parámetros: **dato** y **comando** de tipo entero. Este servicio se utilizará para llenar la tabla con los artistas registrados y el registro de un artista acorde a su número de documento.

```
[ HttpGet( "consultaArtista" ) ]
public IQueryable Get( int comando, int dato ) {
  clsOpeArtista oX = new clsOpeArtista( ctx );
  switch ( comando ) {
    case 1:  // Consultar un artista por nro. de documento
      return oX.artistaXNroDoc( dato );
    case 2: // Llenar tabla
      return oX.listarArtistas( );
    default:
      return Enumerable.Empty<string>().AsQueryable();
  }
}
```

c. Ejecutar el proyecto y probar el servicio con el software: **Swagger** que se abre al ejecutar la API.

i. Recuperar la lista de los datos de un estudiante a partir del número de documento al invocar el método: **consultaArtista.** Por ejemplo, con los siguientes valores: comando = 1 y dato = 9890, mostrará los datos de: `Mauricio Cardona`

ii. Recuperar la lista de todos los estudiantes al invocar el método: **consultaArtista** (llenar tabla). Por ejemplo, con los siguientes valores: comando = 2 y dato = 0, mostrará los datos de todos los artistas registrados.

5.2.2. En el archivo **JavaScript** del formulario crear los procedimientos para buscar un artista por número de documento.

a. Consultar o buscar un artista

i. Crear la function asincrónica: **Consultar( )**

```
async function Consultar( ) {
  try {
    let nroDoc = $("#txtNroDoc").val();
```



Powered by CamScanner

```
        Cancelar();
        $("#txtNroDoc").val(nroDoc);
        if ( isNaN(nroDoc) || parseFloat(nroDoc, 10) <= 0 || nroDoc.trim() == "" ) {
          alerta("Error, el nro. del documento no es válido", "Error en procedimiento");
          $("#txtNroDoc").trigger("focus");
          return;
        }
        let url = dir + "artista/consultaArtista?comando=1&dato=" + nroDoc;
        const datosOut = await fetch( url, {
          method: "GET",
          mode: "cors",url
          headers: {
            "content-type": "application/json",
          }
        } );
        const datosIn = await datosOut.json();
        if ( datosIn == "" ) {
          alerta( "No se encontró artista con nro. doc.: " + nroDoc, "Información del proceso" );
          return;
        }
        $("#txtCodigo").val(datosIn[0].codigo);
        $("#txtNombre").val(datosIn[0].nombre);
        $("#cboGenero").val(datosIn[0].idG);
        $("#cboTipDoc").val(datosIn[0].idTD);
        $("#cboDpto").val(datosIn[0].idDep);
        // Llenar Combo Ciudad y seleccionar ciudad
        await llenarComboCiudad( datosIn[0].idC );
        $("#chkActivo").prop("checked", datosIn[0].act);
      }
      catch (error) {
        alerta("Error: " + error, "Error en consultar");
      }
    }
```

ii. Invocar el listener para el botón **Consultar**. En la opción: $("**#btnBusc**").on("click", invocar la function asincrónica: **Consultar**().

b. Llenar tabla o datatable

i. En el encabezado del archivo (antes de jQuery), definir una variable de tipo dataTable, con la tabla que se quiere procesar

```
var oTabla = $("#tblDatos").DataTable( );
```

ii. Crear la function async para llenar la tabla

```
async function llenarTabla( ) {
  let url = dir + "artista/consultaArtista?comando=2&dato=0";
  let rpta = await llenarTablaGral( url, "#tblDatos");
  if ( rpta != "Termino" ) {
    alerta( "Error en llenado de tabla", "Error en procedimiento" );
  }
}
```

iii. Dentro del jQuery <u>después de los llenados de combos</u>, invocar el llenado de la tabla.

```
llenarTabla( );
```

iv. Crear la function: **editarFila**, para realizar la consulta y llenado de los objetos del formulario con los datos del estudiante, acorde a la fila seleccionada

```
//Buscar desde la tabla
function editarFila( datosFila ) {
  let nroDoc = datosFila.find('td:eq(4)').text()
  if ( nroDoc > 0 ) {
    $("#txtNroDoc").val(nroDoc);
    Consultar();
    alerta( "Ok", "Procedimiento con éxito" );
    return;
  }
  alerta("Sin registros", "Procedimiento incompleto");
}
```

v. Crear el listener para la tabla antes del final del jQuery para el evento clic en un registro (tr) de la tabla

```
//Listener el datatable
$("#tblDatos tbody").on( "click", 'tr', function ( evento ) {
  evento.preventDefault( );  //Esto permite al desarrollador tomar el control del comportamiento del evento y realizar acciones personalizadas
  //Levanta el evento del clic sobre la tabla
  if ($(this).hasClass('selected')) {
    $(this).removeClass('selected');
  }
  $('#tblDatos tbody tr.selected').removeClass('selected');
  $(this).addClass('selected');
  editarFila($(this).closest('tr'));
} );
```



Powered by CamScanner

5.2.3. En el archivo **HTML** del formulario.

i. En la sección del Head, observar los scripts de diseño desde la carpeta: **Estilos**

```
<!--Estilos para las tablas-->
<link href="../Estilos/dataTables.min.css" rel="stylesheet" />
<link href="../Estilos/dataTables.bootstrap4.min.css" rel="stylesheet" />
```

ii. Al final de la página antes del fin del cuerpo **</body>**, observar los scripts de control y llenado de la tabla:

```
<!-- Tabla -->
<script src="../Scripts/jquery.dataTables.min.js"></script>                    En: [Scripts]
<script src="../Scripts /dataTables.bootstrap4.min.js"></script>               En: [Scripts]
<!--Librerías comunes de JavaScript para las Tablas -->
<script src="Scripts_Form /Comunes/Tablas.js"></script>                        En: [Scripts_Form/Comunes]
```

**6. Agregar/Insertar**

6.1. En la clase operativa **clsOpeArtista**.

i. Crear la propiedad: **tblArtis** de tipo tabla **TblArtistum** del Entity framework de entrada y de salida.

```
public TblArtistum tblArtis { get; set; }
```

ii. Crear el método **Agregar** para adicionar la información de un artista acorde a los datos llegados desde un archivo Json de tipo **tblArtista**, retorna un valor entero: 0 : Error de ejecución, -1: Ya existe un registro o el número de la clave primaria del registro ingresado.

```
public int Agregar( ) {
  try
  {  // Validar si existe
     var existe = ctx.TblArtista.FirstOrDefault( x => x.NroDoc == tblArtis.NroDoc );
     if ( existe != null )
        return -1;  //Ya existe
     else {   // Hallar el código
        int newCod = ctx.TblArtista.Max( x => x.Codigo );
        tblArtis.Codigo = ( newCod <= 0 ) ? 1 : newCod + 1;
        ctx.TblArtista.Add( tblArtis );
        ctx.SaveChangesAsync( );
        return newCod; //Código como éxito
     }
  }
  catch {
     return 0;  //Error de ejecución
  }
}
```

iii. Crear el método **Modificar** para modificar/actualizar la información de un artista acorde a los datos llegados desde un archivo Json de tipo **tblArtista**, retorna un valor entero: 0 : Error de ejecución, -1: No existe un registro o el número de la clave primaria del registro modificado.

```
public int Modificar( )  {
  try
  {  // Validar si existe
     var rgtro = ctx.TblArtista.FirstOrDefault( x => x.Codigo == tblArtis.Codigo );
     if ( rgtro == null ) //Si no existe
        return -1;  //Error, no existe
     // Actualiza cada dato
     rgtro.Nombre = tblArtis.Nombre;
     rgtro.IdtipoDoc = tblArtis.IdtipoDoc;
     rgtro.NroDoc = tblArtis.NroDoc;
     rgtro.IdGenero = tblArtis.IdGenero;
     rgtro.IdCiudad = tblArtis.IdCiudad;
     rgtro.Activo = tblArtis.Activo;
     ctx.SaveChanges( );
     return tblArtis.Codigo;
  }
  catch
  {
     return 0;
  }
}
```

6.2. En la clase controladora **artistaController**.

i. Crear la propiedad: **tblArtis** de tipo tabla **TblArtistum** del Entity framework de entrada y de salida.

```
public TblArtistum tblArtis { get; set; }
```

ii. Crear el servicio para agregar un registro utilizando el método **HttpPost Post** con parámetro de tipo Json de la tabla **TblArtistum** del Entity framework para agregar/insertar un nuevo artista utilizando el método: **Agregar** de la clase operativa.

```
[HttpPost]
public int Post( [FromBody] TblArtistum datIn )
{  //Crear un objeto desde la clase operativa
   clsOpeArtista oX = new clsOpeArtista( ctx );
   oX.tblArtis = datIn; //Pasar los parámetros del estudiante
   return oX.Agregar( );
}
```



Powered by CamScanner

iii. Crear el servicio para modificar un registro utilizando el método **HttpPut Put** con parámetro de tipo Json de la tabla **TblArtistum** del Entity framework para modificar un artista utilizando el método: **Modificar** de la clase operativa.

```
[HttpPut]
public int Put( [FromBody] TblArtistum datIn )
{   //Crear un objeto desde la clase operativa
    clsOpeArtista oX = new clsOpeArtista( ctx );
    oX.tblArtis = datIn; //Pasar los parámetros del estudiante
    return oX.Modificar( );
}
```

6.3. Ejecutar el proyecto y probar el servicio con el software: **Swagger** que se abre al ejecutar la API. Ejecutar

i. Crear el formato Json a continuación para adicionar un nuevo registro a la tabla: **TblArtista.**

```
{
 "codigo"    : 0,
 "nombre"    : "El Chavo del 8",
 "idtipoDoc" : 1,
 "nroDoc"    : 12345,
 "idGenero"  : 1,
 "idCiudad"  : 2,
 "activo"    : true
}
```

ii. Crear el formato Json a continuación para modificar un registro existente en la tabla: **TblArtista**. Asumiendo que en el ejemplo anterior retorno como código del estudiante el número: 13, en negrilla los datos a modificar.

```
{
 "codigo"    : 13,
 "nombre"    : "El Chavito del 8",
 "idtipoDoc" : 2,
 "nroDoc"    : 123456,
 "idGenero"  : 1,
 "idCiudad"  : 3,
 "activo"    : false
}
```

Realizar la consulta de un estudiante: **consultaArtista** con el **comando**: 1 y **dato**: 123456

6.4. En el archivo **JavaScript** del formulario

i. Modificar la función sincrónica validar para realizar la validación del código, el nombre y el número del documento del artista con el siguiente código:

```
function validar( name, nroDoc ) {
   var msj = "Falta ";
   if ( name.trim() == "" ) {
      msj = msj + "El nombre no válido, \n";
   }
   if ( isNaN( nroDoc ) || parseFloat( nroDoc, 10 ) <= 0 || nroDoc.trim( ) == "" ) {
      msj = msj + "el nro. de documento no válido. \n";
   }
   if ( msj != "Falta " ) {
      alerta(msj, "Error en procedimiento" );
      return false;
   }
   return true;
}
```

ii. Modificar la function **ejecutarComando**, para ser utilizada tanto para **agregar** y para **modificar** de la siguiente manera:

```
async function ejecutarComando( acción ) {
   //Capturar los datos de entrada
   let Codi = $("#txtCodigo").val();
   let Nomb = $("#txtNombre").val();
   let idGe = $("#cboGenero").val();
   let idTD = $("#cboTipDoc").val();
   let nroD = $("#txtNroDoc").val();
   let idCi = $("#cboCiudad").val();
   let acti = $("#chkActivo").prop("checked");
   alerta("Codi: " + Codi + ", Nomb: " + Nomb + ", idGe: " + idGe + ", idTD: " + idTD + ", nroD: " + nroD + ", idCi: " + idCi + ",
activo: " + acti , "Datos obtenidos" );
   // Validar información
   if ( ! validar( Nomb, nroD ) ) {
      $("#txtNombre").trigger("focus");
      return;
   }
   // Confirmar Procedimiento
   let rpta = "";
   if ( accion === "POST" ) {
      if ( isNaN(Codi) || parseInt( Codi, 10 ) != 0 || Codi.trim() == "" ) {
         alerta("Código no válido para registro nuevo", "Alerta");
         return;
      }
```



Powered by CamScanner

```
        rpta = window.confirm('Estas seguro de agregar datos de: ' + Nomb + ', con nro. Doc.: ' + nroD + '?');
        if ( rpta == false ) {
            alerta( 'Cancelada acción de Agregar', 'Información' );
            return;
        }
    } else if ( accion === "PUT" ) {
        if ( isNaN(Codi) || parseInt( Codi, 10 ) <= 0 || Codi.trim() == "" ) {
            alerta("Código no válido para modificar registro", "Alerta");
            return;
        }
        rpta = window.confirm('Estas seguro de modificar datos de: ' + Nomb + ', con nro. Doc.: ' + nroD + '?');
        if ( rpta == false ) {
            alerta( 'Cancelada acción de Insertar', 'Información' );
            return;
        }
    }
    else {
        alerta("Procedimiento no válido", "Error");
        return;
    }
    //Crear JSon
    let datosOut = {
        codigo   : Codi,
        nombre   : Nomb,
        idtipoDoc: idTD,
        nroDoc   : nroD,
        idGenero : idGe,
        idCiudad : idCi,
        activo   : acti
    }
    //Invocar el servicio con fetch
    try {      // fetch se utiliza para realizar solicitudes HTTP a recursos de red, como obtener datos de un servidor o enviar datos a un servidor.
        const response = await fetch( dir + "artista", {
            method: accion,
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify( datosOut ),
        });  // stringify( ) convierte un objeto o valor de JavaScript en una cadena de texto JSON

        const rpta = await response.json( );
        if ( rpta == -1 )
            alerta("Problema al grabar el registro (Revisar si existe)", "Error en proceso");
        else if ( rpta == 0 )
            alerta("Error al momento del grabar, reintente por favor", "Error en grabar");
        else if (rpta > 0) {
            alerta("Registro grabado con éxito", "Proceso con éxito");
            $("#txtCodigo").val( rpta ); //Cambiar por texto que contenga la clave primaria del registro
            await Consultar( );
            await llenarTabla( );
        }
        else
            alerta( rpta, "Error en procedimiento", "Error" );
    } catch (error) {
        alerta( "Error: " + error, "Error en procedimiento" );
    }
}
```

iii. Invocar el listener para el botón **Agregar**. En la opción: $("#btnAgre ").on("click", agregar el siguiente código:

```
$("#txtCodigo").val("0");
ejecutarComando( "POST" );
```

iv. Invocar el listener para el botón **Modificar**. En la opción: $("#btnModi ").on("click", agregar el siguiente código:

```
let cod = $("#txtCodigo").val();
 if ( isNaN( cod ) || parseInt( cod, 10 ) <= 0 || cod.trim( ) == "" ) {
    alerta("Error, procedimiento no válido.", "Error, Buscar primero un artista");
    return;
 }
ejecutarComando("PUT");
```



8. Tema de estu

Entity Framework
Es un conjunto de tec
arquitectos y programa
diferentes. Deben mod
deben trabajar con los

Powered by CamScanner

## 8. Tema de estudio

### Entity Framework

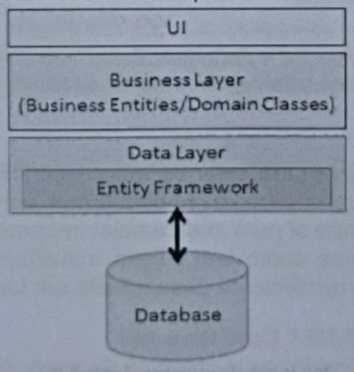
Es un conjunto de tecnologías de ADO.NET que permiten el desarrollo de aplicaciones de software orientadas a datos. Los arquitectos y programadores de aplicaciones orientadas a datos se han enfrentado a la necesidad de lograr dos objetivos muy diferentes. Deben modelar las entidades, las relaciones y la lógica de los problemas empresariales que resuelven, y también deben trabajar con los motores de datos que se usan para almacenar y recuperar los datos. Los datos pueden abarcar varios sistemas de almacenamiento, cada uno con sus propios protocolos; incluso las aplicaciones que funcionan con un único sistema de almacenamiento deben equilibrar los requisitos del sistema de almacenamiento con respecto a los requisitos de escribir un código de aplicación eficaz y fácil de mantener.

Entity Framework permite a los programadores trabajar con datos en forma de objetos y propiedades específicos del dominio, por ejemplo, con clientes y direcciones de clientes, sin tener que pensar en las tablas de las bases de datos subyacentes y en las columnas en las que se almacenan estos datos. Con Entity Framework, los desarrolladores pueden trabajar en un nivel más alto de abstracción cuando tratan con datos, y pueden crear y mantener aplicaciones orientadas a datos con menos código que en las aplicaciones tradicionales. Dado que Entity Framework es un componente de .NET Framework, las aplicaciones de Entity Framework se pueden ejecutar en cualquier equipo en el que esté instalado .NET Framework a partir de la versión 3.5 SP1.

Antes de .NET 3.5, nosotros (los desarrolladores) solíamos escribir código ADO.NET o Enterprise Data Access Block para guardar o recuperar datos de la aplicación de la base de datos subyacente. Solíamos abrir una conexión a la base de datos, crear un conjunto de datos para recuperar o enviar los datos a la base de datos, convertir datos del conjunto de datos en objetos .NET o viceversa para aplicar reglas comerciales. Este fue un proceso engorroso y propenso a errores. Microsoft ha proporcionado un marco llamado "Entity Framework" para automatizar todas estas actividades relacionadas con la base de datos para su aplicación.

Entity Framework es un marco ORM Object–relational mapping (ORM, O/RM, and O/R mapping tool) (Mapeo Relacional de Objetos) de código abierto para aplicaciones .NET compatibles con Microsoft. Permite a los desarrolladores trabajar con datos utilizando objetos de clases específicas de dominio sin centrarse en las tablas y columnas de la base de datos subyacente donde se almacenan estos datos. Con Entity Framework, los desarrolladores pueden trabajar en un mayor nivel de abstracción cuando manejan datos y pueden crear y mantener aplicaciones orientadas a datos con menos código en comparación con las aplicaciones tradicionales.

Definición oficial: "Entity Framework es un asignador relacional de objetos (O/RM) que permite a los desarrolladores de .NET trabajar con una base de datos utilizando objetos .NET. Elimina la necesidad de la mayor parte del código de acceso a datos que los desarrolladores normalmente necesitan escribir".
La siguiente figura ilustra dónde encaja Entity Framework en su aplicación.




Según la figura anterior, Entity Framework encaja entre las entidades comerciales (clases de dominio) y la base de datos. Guarda los datos almacenados en las propiedades de las entidades comerciales y también recupera datos de la base de datos y los convierte automáticamente en objetos de entidades comerciales.

### Características del marco de entidad

- **Multiplataforma**: EF Core es un marco multiplataforma que puede ejecutarse en Windows, Linux y Mac.
- **Modelado:** EF (Entity Framework) crea un EDM (Entity Data Model) basado en entidades POCO (Plain Old CLR Object) con propiedades get/set de diferentes tipos de datos. Utiliza este modelo al consultar o guardar datos de entidad en la base de datos subyacente.
- **Consultas:** EF nos permite utilizar consultas LINQ (C#/VB.NET) para recuperar datos de la base de datos subyacente. El proveedor de la base de datos traducirá estas consultas LINQ al lenguaje de consulta específico de la base de datos (por ejemplo, SQL para una base de datos relacional). EF también nos permite ejecutar consultas SQL sin formato directamente a la base de datos.



Powered by CamScanner

- **Seguimiento de cambios:** EF realiza un seguimiento de los cambios ocurridos en las instancias de sus entidades (valor de propiedad) que deben enviarse a la base de datos.
- **Guardar:** EF ejecuta los comandos INSERTAR, ACTUALIZAR y ELIMINAR en la base de datos en función de los cambios ocurridos en sus entidades cuando llama al SaveChanges()método. EF también proporciona el SaveChangesAsync()método asincrónico.
- **Simultaneidad:** EF utiliza la simultaneidad optimista de forma predeterminada para proteger la sobrescritura de cambios realizados por otro usuario desde que se obtuvieron los datos de la base de datos.
- **Transacciones:** EF realiza una gestión automática de transacciones mientras consulta o guarda datos. También proporciona opciones para personalizar la gestión de transacciones.
- **Almacenamiento en caché:** EF incluye el primer nivel de almacenamiento en caché listo para usar. Por lo tanto, las consultas repetidas devolverán datos del caché en lugar de llegar a la base de datos.
- **Convenciones integradas:** EF sigue convenciones sobre el patrón de programación de configuración e incluye un conjunto de reglas predeterminadas que configuran automáticamente el modelo de EF.
- **Configuraciones:** EF nos permite configurar el modelo EF mediante el uso de atributos de anotación de datos o Fluent API para anular las convenciones predeterminadas.
- **Migraciones:** EF proporciona un conjunto de comandos de migración que se pueden ejecutar en la consola del administrador de paquetes NuGet o en la interfaz de línea de comandos para crear o administrar el esquema de la base de datos subyacente.

https://www.entityframeworktutorial.net/what-is-entityframework.aspx

**Diferencia de IEnumerable e IQueryable**

Los **IEnumerable** se deberían emplear en consultas de objetos en memoria, como listas y arreglos, por ejemplo. Ahora bien, si nuestra consulta **Linq** fuese a una base de datos y usáramos un **IEnumerable**, lo que sucedería es que se traerían de la base de datos todos los registros, estos se cargarían en memoria y sería entonces donde se aplicarían los filtros de la consulta, dígase lo que va en el **Where**. Esto es ineficiente, ya que estaríamos gastando más memoria de la necesaria.
Por otra parte, los **IQueryables** implementan la **Interfaz IEnumerable**, por tanto, todo lo que se hace con un **IEnumerable** se puede hacer con **IQueryable**. El **IQueryable** ejecuta los filtros en el mismo servidor de base de datos (estaría en una máquina diferente a la nuestra), y trae los datos ya filtrados a nuestra computadora.
Resumen, usar **IEnumerable** para consultas locales y **IQueryable** para consultas remotas. Hay más diferencias, pero está a mi entender es la más importante.

Usando **IEnumerable**:

```
IEnumerable<Estudiante> list = dbContext.Estudiantes.Where(e => e.Nota > 80);
```

Se traerían todos los registros de la **tabla Estudiante** se cargarían en la memoria local y entonces aquí sería donde se ejecutaría el **Where**.

Usando **IQueryable**:

```
IQueryable<Estudiante> list = dbContext.Estudiantes.Where(e => e.Nota > 80);
```

En la misma base de datos se ejecuta el **Where** y los registros vienen ya filtrados a nuestra computadora.

**Proyecto ASP.NET Core Web API**

Un proyecto ASP.NET Core Web API sirve para crear servicios web RESTful, es decir, aplicaciones que exponen datos y funcionalidades de forma estandarizada a través de Internet, permitiendo que diferentes aplicaciones (web, móviles, de escritorio) se comuniquen entre sí para intercambiar información. Utiliza los verbos HTTP (GET, POST, PUT, DELETE) para realizar operaciones como leer, crear, actualizar y eliminar datos, y es una herramienta multiplataforma, de código abierto y alto rendimiento desarrollado por Microsoft.

**Función principal de un proyecto ASP.NET Core Web API**

- ✓ **Servicios RESTful:** Su objetivo principal es la creación de servicios web que siguen las directrices del estilo arquitectónico REST.
- ✓ **Comunicación entre aplicaciones:** Permite la comunicación entre aplicaciones de diferentes plataformas (navegadores, dispositivos móviles, etc.) mediante Internet.
- ✓ **Intercambio de datos:** Facilita el intercambio de información, típicamente en formatos como JSON o XML, entre el servidor y otros clientes.
- ✓ **Acciones CRUD:** Implementa las operaciones básicas de datos (Crear, Leer, Actualizar, Eliminar) utilizando los métodos HTTP correspondientes (POST, GET, PUT, DELETE).

**¿Cómo funciona?**

1. **Servidor y Clientes:** Un proyecto Web API actúa como el servidor que expone sus datos y lógica de negocio a través de una interfaz (la API).
2. **Verbos HTTP:** Los clientes (como un navegador o una aplicación móvil) envían solicitudes a la API utilizando verbos HTTP.



Powered by CamScanner